

Aplicação FastAPI

Da instalação à primeira API

```
pip install fastapi  
fastapi dev main.py  
fast
```

Criar aplicação API RESTFUL com fastapi
com GET e POST

O que é uma API?

API — Application Programming Interface permite que diferentes aplicações se comuniquem.



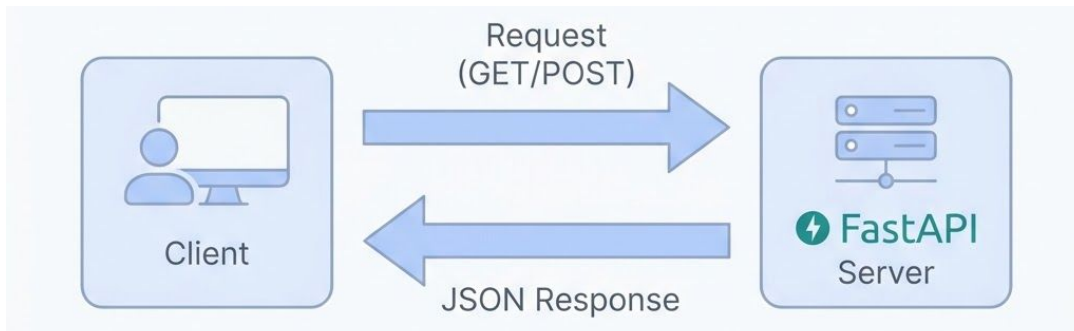
Frontend

- Frontend não precisa saber como os produtos estão armazenados.
- Ele apenas precisa saber como conversar com a API.



Cliente/Servidor

- Cliente solicita:
 - GET /produtos
- Servidor responde:
 - [{ "id": 1, "nome": "Notebook", "preco": 3500 }, { "id": 2, "nome": "Mouse", "preco": 100 }]



Métodos HTTP



GET

- Consultar dados
 - Exemplo: listar produtos



POST

- Criar dados
 - Exemplo: cadastrar produto



PUT

- Atualizar um recurso
 - Exemplo: alterar produto



DELETE

- Excluir um recurso
 - Exemplo: remover produto

Métodos HTTP



GET

- Consultar dados
 - Exemplo: listar produtos



POST

- Criar dados
 - Exemplo: cadastrar produto



PUT

- Atualizar um recurso
 - Exemplo: alterar produto



DELETE

- Excluir um recurso
 - Exemplo: remover produto

Primeira aplicação

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"mensagem": "Minha primeira API"}
```



Importando FastAPI

- Importando a classe principal do framework.



Criando a aplicação

- app representa a aplicação
- Nesse objeto são registrados os endpoints.



Criando uma rota

- `@app.get("/")` informa ao FastAPI:
 - “Quando alguém fizer uma requisição GET para /, execute a função abaixo”
- Decorator associa uma rota e um método HTTP a uma função Python.



Criando a função

- `def home() :`
 - `return {"mensagem": "Minha primeira API"}`
 - Quando o endpoint for acessado, a função será executada.
- `{"mensagem": "Minha primeira API"}`
 - Enviado ao cliente como uma resposta JSON.

Executando a aplicação



No terminal

- `fastapi dev main.py`



No navegador

- `http://127.0.0.1:8000`



Resposta no navegador

- `{ "mensagem": "Minha primeira API" }`



Swagger

- <http://127.0.0.1:8000/docs> abre uma interface de acesso a API chamada swagger

Criando um recurso: produtos

```
produtos = [  
  {  
    "id": 1,  
    "nome": "Notebook",  
    "preco": 3500.0  
  },  
  {  
    "id": 2,  
    "nome": "Mouse",  
    "preco": 100.0  
  }  
]
```



Produtos

- Essa lista funcionará temporariamente como nosso "banco de dados".
- Em uma aplicação real, provavelmente utilizaríamos um banco de dados

Criando um endpoint para listar produtos

```
@app.get("/produtos")
def listar_produtos():
    return produtos
```



GET /produtos ou <http://127.0.0.1:8000/produtos>

- Resultado:
 - [
 - {
 - "id": 1,
 - "nome": "Notebook",
 - "preco": 3500.0
 - },
 - {
 - "id": 2,
 - "nome": "Mouse",
 - "preco": 100.0
 - }
 -]

Parâmetros de rota

```
@app.get("/produtos/{produto_id}")
def buscar_produto(produto_id: int):

    for produto in produtos:
        if produto["id"] == produto_id:
            return produto

    return {"erro": "Produto não encontrado"}
```



GET /produtos/1 ou http://127.0.0.1:8000/produtos/1

- produto_id: int indica que produto_id deve ser um inteiro
- FastAPI reconhece {produto_id} como parâmetro da rota e utiliza a anotação int para converter e validar o valor recebido
- Resultado:
 - {
 "id": 1,
 "nome": "Notebook",
 "preco": 3500.0
}

Retornando erros HTTP

```
@app.get("/produtos/{produto_id}")
def buscar_produto(produto_id: int):

    for produto in produtos:
        if produto["id"] == produto_id:
            return produto

    raise HTTPException(
        status_code=404,
        detail="Produto não encontrado"
    )
```



**GET /produtos/abc ou
http://127.0.0.1:8000/produtos/abc**

- Produto abc não existe
- Agora, quando um produto não existir, a API retorna:
 - 404 Not Found
- Resultado:
 - { "detail": "Produto não encontrado" }

Query parameters

```
@app.get("/listar_produtos")
def listar_produtos(limite: int = 10):
    return produtos[:limite]
```



GET /produtos?limite=1

- Limite não aparece dentro da rota: "/produtos"
 - FastAPI interpreta esse argumento como um query parameter.
- Resultado:
 - [
 - {
 - "id": 1,
 - "nome": "Notebook",
 - "preco": 3500.0

Criando dados com POST

```
@app.post("/produtos", status_code=201)
async def criar_produto(request: Request):

    produto = await request.json()

    novo_produto = {
        "id": len(produtos) + 1,
        "nome": produto["nome"],
        "preco": produto["preco"]
    }
    produtos.append(novo_produto)

    return novo_produto
```



POST /produtos

- Cliente envia:
 - {
 "nome": "Teclado",
 "preco": 250
}
- Cliente recebe:
 - {
 "id": 3,
 "nome": "Teclado",
 "preco": 250
}



Método request.json()

- Lê o corpo da requisição e o converte para um dicionário Python.

Testando o POST com requests

```
import requests

url1 =
"http://127.0.0.1:8000/criar_produto"
url2=
"http://127.0.0.1:8000/listar_produtos"
url3=
"http://127.0.0.1:8000/listar_produtos?lim
ite=4"
novo_produto = {
    "nome": "Teclado",
    "preco": 4000.0
}
response = requests.post(url1,
json=novo_produto)
response = requests.post(url1,
json=novo_produto)
response = requests.get(url3)
print(response.json())
```



Instalar requests

- pip install requests



requests.get e requests.post

- Realizam requisições GET e POST HTTP
 - request.get: apenas url
 - request.post: url e conteúdo no formato json a ser enviado



Outro Terminal

- python test.py
 - Se o arquivo se chamar test.py

Modelos Pydantic

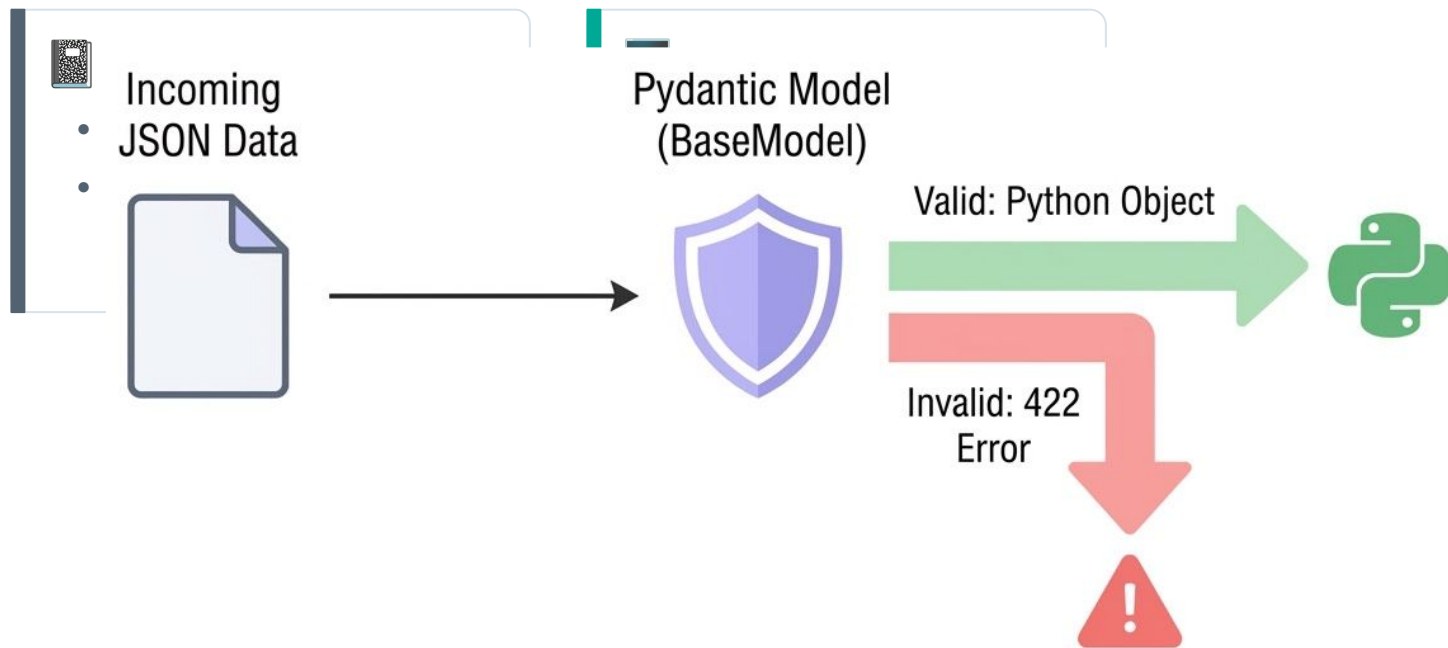
Da instalação à primeira API

```
pip install fastapi  
fastapi dev main.py  
fast
```

Criar aplicação API RESTFUL com fastapi
com GET e POST

Modelos Pydantic

Pydantic: biblioteca Python utilizada para trabalhar com dados estruturados e validação baseada em tipos.



Modelo Pydantic



Definição do modelo

- ```
class Produto(BaseModel):
 nome: str
 preco: float
```



## Atributos esperados

- Um Produto possui:
  - nome → str
  - preco → float



# Importando BaseModel

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Produto(BaseModel):
 nome: str
 preco: float
```



## Produto

- Classe chamada Produto que herda BaseModel
- Formato esperado dos dados:
  - nome: str
  - preco: float
- Produto válido:
  - {  
    "nome": "Notebook",  
    "preco": 3500  
}
- Produto inválido:
  - {  
    "nome": 1000,  
    "preco": 3500  
}

# Criando objetos Pydantic em Python

```
from pydantic import BaseModel
```

```
class Produto(BaseModel):
 nome: str
 preco: float
```

```
produto = Produto(
 nome="Notebook",
 preco=3500
)
```

```
dados = produto.model_dump()

print(dados)
```



## Objeto Pydantic Produto

- Acesso aos seus atributos:
  - `print(produto.nome)`  
`print(produto.preco)`



## `produto.model_dump()`

- Converte um modelo para dicionário
- Resultado:
  - `{`  
    `"nome": "Notebook",`  
    `"preco": 3500.0`  
    `}`

# Utilizando o modelo no FastAPI

```
@app.post("/produtos", status_code=201)
def criar_produto(produto: Produto):

 novo_produto = {
 "id": len(produtos) + 1,
 "nome": produto.nome,
 "preco": produto.preco
 }

 produtos.append(novo_produto)

 return novo_produto
```



## produto: Produto

- FastAPI percebe que Produto é um modelo Pydantic
  - Dados devem vir no corpo da requisição e devem possuir a estrutura definida por esse modelo.
- FastAPI realiza esse trabalho automaticamente.

# Validação automática



## Validação do modelo

- Classe:
  - `class Produto(BaseModel):`  
    `nome: str`  
    `preco: float`
- Envio 1:
  - `{`  
    `"nome": "Teclado",`  
    `"preco": "abc"`  
    `}`
- Envio 2:
  - `{`  
    `"nome": "Teclado"`  
    `}`



## Erro retornado

- Envio1:
  - O valor "abc" não pode ser utilizado como um número float
- Envio 2:
  - Os dois campos são obrigatórios
- FastAPI rejeitará automaticamente a requisição antes mesmo da execução da nossa função.

# Adicionando um novo campo

```
class Produto(BaseModel):
 nome: str
 preco: float
 categoria: str
```



## Corpo esperado

- ```
{  
  "nome": "Teclado",  
  "preco": 250,  
  "categoria": "Periféricos"  
}
```
- Alterar modelo já modifica o formato esperado pela API

Campos opcionais e valores padrão

```
class Produto(BaseModel):  
    nome: str  
    preco: float  
    categoria: str  
    descricao: str | None = None  
    disponivel : bool = True
```

JSONs válidos

- {
 "nome": "Teclado",
 "preco": 250,
 "categoria": "Periféricos"
}
- {
 "nome": "Teclado",
 "preco": 250,
 "categoria": "Periféricos",
 "descricao": "Teclado mecânico"
}



Campos opcionais

- | None = None indica que o campo é opcional
- Opcionais:
 - descricao



Campos padrão

- var : type = value indica que value é padrão para var
- Valor padrão é opcional
- Padrão:
 - disponivel = True

Validações com Field

```
from pydantic import BaseModel, Field
class Produto(BaseModel):
    nome: str = Field(min_length=2)
    preco: float = Field(gt=0 )
    categoria: str
    descricao: str | None = None
    disponivel : bool = True
```



Validações de campos

- Field(gt=0) significa greater than 0 ou maior que zero
- Field(min_length=2) significa que o nome precisa ter pelo menos dois caracteres

Dica: Validações com Fields

<https://pydantic.dev/docs/validation/latest/concepts/fields/>